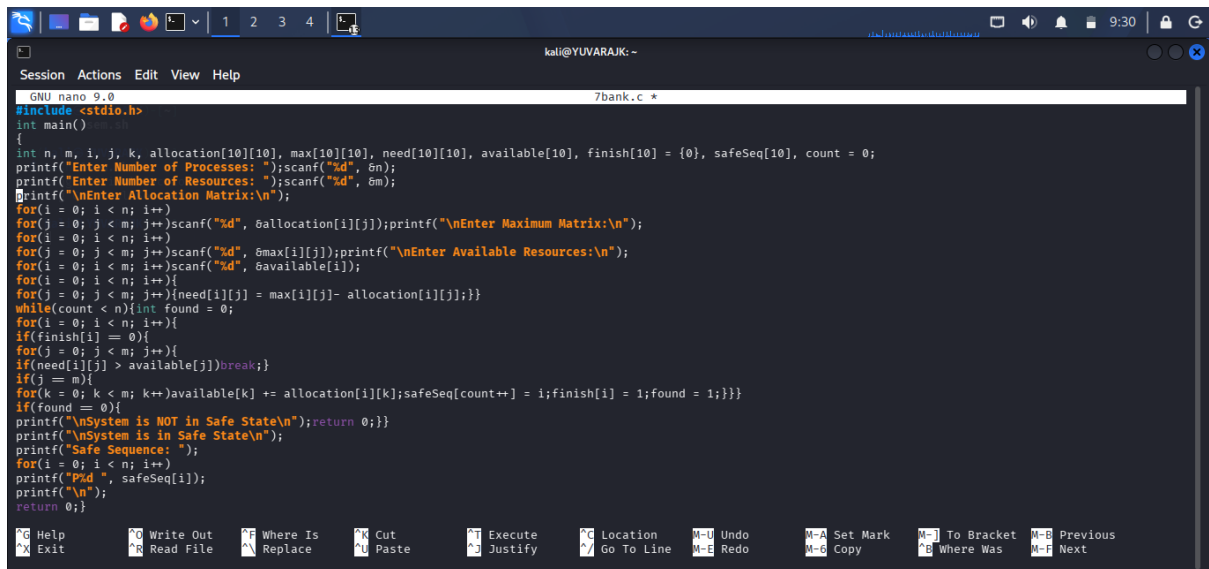
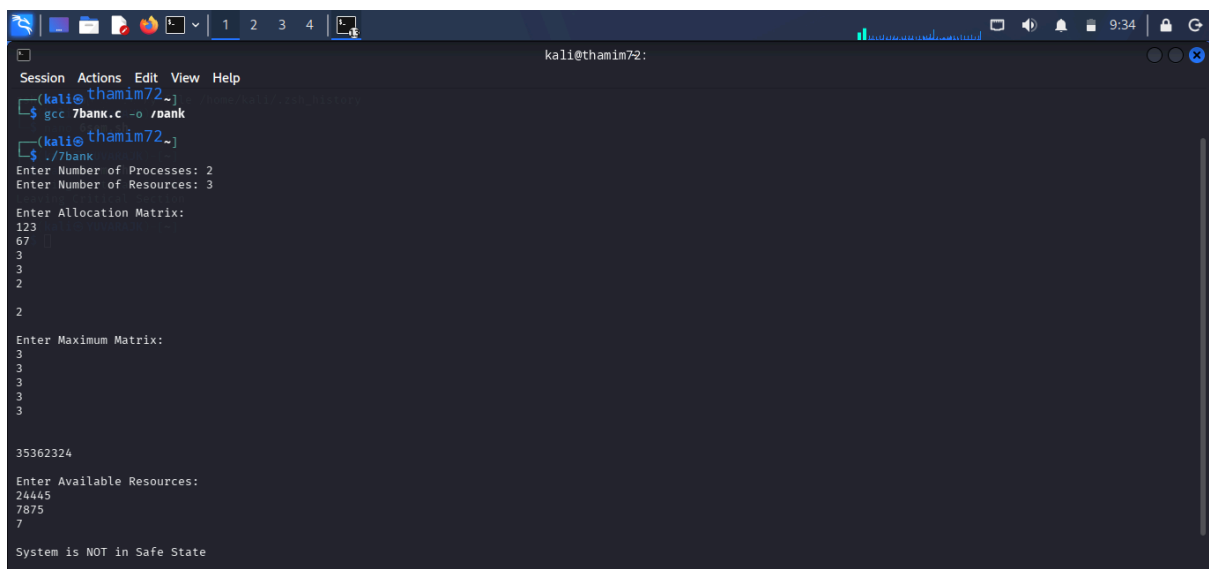


C PROGRAMMING :

A screenshot of a terminal window showing a C program being edited in the nano text editor. The program is named '7bank.c' and implements a deadlock detection algorithm. It takes user input for the number of processes (n) and resources (m), then an allocation matrix. It calculates the maximum matrix and available resources, then checks for a safe sequence. The code includes standard headers, variable declarations, and loops for input, calculation, and checking for a safe state. The terminal window title is 'kali@YUVARAJK: ~' and the nano editor status bar shows various keyboard shortcuts.

```
GNU nano 9.0 7bank.c *
#include <stdio.h>
int main()
{
    int n, m, i, j, k, allocation[10][10], max[10][10], need[10][10], available[10], finish[10] = {0}, safeSeq[10], count = 0;
    printf("Enter Number of Processes: ");scanf("%d", &n);
    printf("Enter Number of Resources: ");scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)scanf("%d", &allocation[i][j]);printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)scanf("%d", &max[i][j]);printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++)scanf("%d", &available[i]);
    for(i = 0; i < n; i++){
        for(j = 0; j < m; j++){
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }
    while(count < n){int found = 0;
        for(i = 0; i < n; i++){
            if(finish[i] == 0){
                for(j = 0; j < m; j++){
                    if(need[i][j] > available[j])break;
                }
                if(j == m){
                    for(k = 0; k < m; k++)available[k] += allocation[i][k];safeSeq[count++] = i;finish[i] = 1;found = 1;
                }
            }
            if(found == 0){
                printf("\nSystem is NOT in Safe State\n");return 0;
            }
            printf("\nSystem is in Safe State\n");
            printf("Safe Sequence: ");
            for(i = 0; i < n; i++)
                printf("%d ", safeSeq[i]);
            printf("\n");
        }
        return 0;
    }
```

OUTPUT :

A screenshot of a terminal window showing the execution of the '7bank.c' program. The user enters 2 for the number of processes and 3 for the number of resources. They then enter an allocation matrix, a maximum matrix, and available resources. The program outputs the safe sequence and concludes that the system is not in a safe state. The terminal window title is 'kali@thamim72: ~' and the prompt is '(kali@thamim72~) \$'.

```
(kali@thamim72~) $ gcc 7bank.c -o 7bank
(kali@thamim72~) $ ./7bank
Enter Number of Processes: 2
Enter Number of Resources: 3

Enter Allocation Matrix:
123
67
3
3
2
2

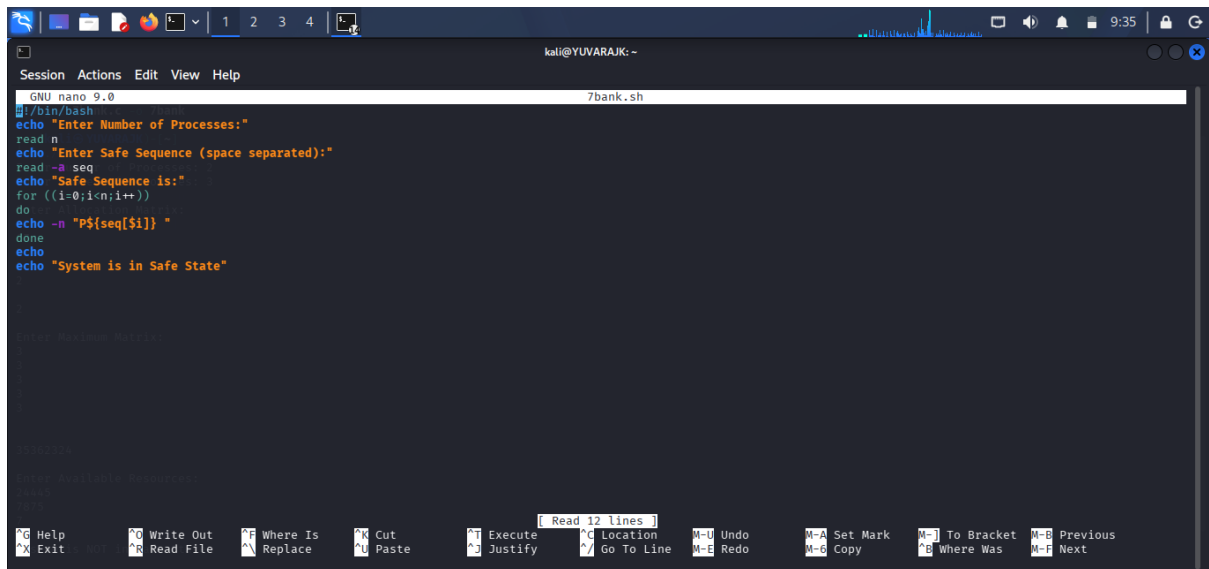
Enter Maximum Matrix:
3
3
3
3
3

35362324

Enter Available Resources:
24445
7875
7

System is NOT in Safe State
```

SHELL PROGRAMMING :

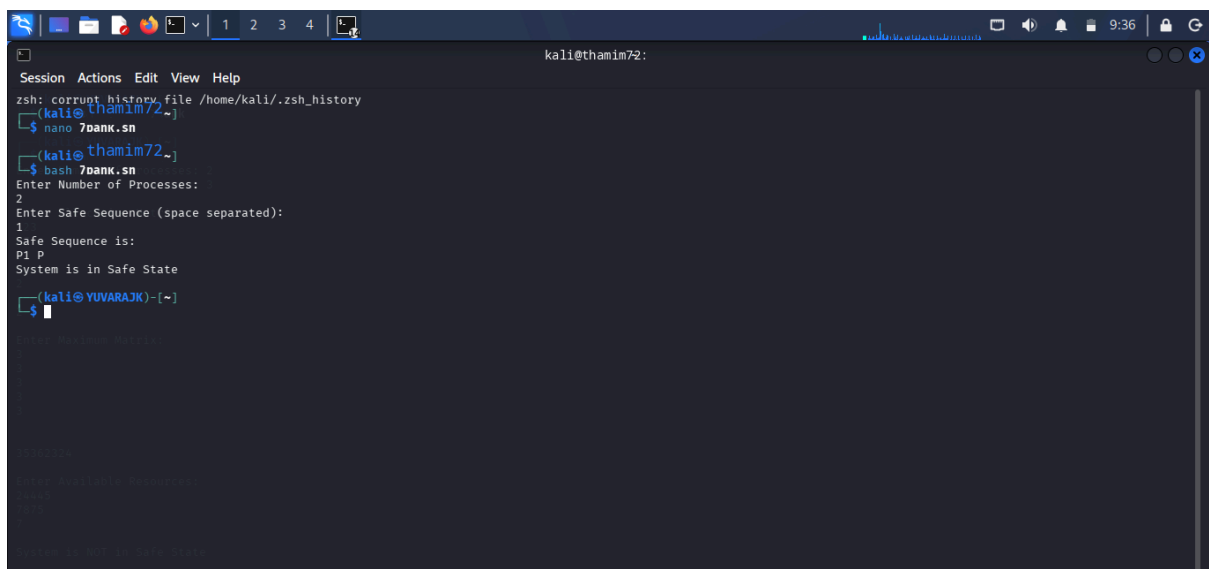


The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 7bank.sh. The script contains the following code:

```
#!/bin/bash
echo "Enter Number of Processes:"
read n
echo "Enter Safe Sequence (space separated):"
read -a seq
echo "Safe Sequence is:"
for ((i=0;i<n;i++))
do
echo -n "${seq[$i]} "
done
echo
echo "System is in Safe State"
```

The nano editor interface includes a menu bar with options: Session, Actions, Edit, View, Help. The status bar at the bottom shows various keyboard shortcuts for editing and execution.

OUTPUT :



The screenshot shows a terminal window with the following output:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@thamim72:~)
$ nano 7bank.sh
(kali@thamim72:~)
$ bash 7bank.sh
Enter Number of Processes:
2
Enter Safe Sequence (space separated):
1
Safe Sequence is:
P1 P
System is in Safe State
(kali@YUVARAJK)-[~]
$
```

The output demonstrates the script's functionality: it prompts for the number of processes (2) and a safe sequence (P1 P), then prints the sequence and confirms the system is in a safe state.